

Expressions, Variables, and Printing

Christopher Martin - *Bowdoin College* [↗](#)

Computers follow instructions to the letter

Computers are really *frustratingly* good at following our instructions **EXACTLY**.

They will do only what we specifically tell them to do - **nothing more, nothing less**.

This means that when we communicate with a computer, we have to be especially mindful about what we are asking them to do and in what order we ask them to do it.

This is - without a doubt - going to be the one aspect of programming you struggle with the most.

Go slow, take your time, and really try to plan out what you are actually "saying".

Instructions

There are two different kind of instructions we can issue to the computer.

Expressions are instructions that are evaluated (like in math) to produce a value or answer. Anything that evaluates to a value is considered an expression. (Even a single value!)

Statements are instructions that are executed to perform an action or task.

Arithmetic Operators

One thing that computers are especially good at is *math*.

Some of the most fundamental things we ask computers to do is to perform simple arithmetic using the **operators** we are familiar with.

Human Operator	Python Symbol	Example
Addition	<code>+</code>	<code>5 + 5 = 10</code>
Subtraction	<code>-</code>	<code>2 - 7 = -5</code>
Multiplication	<code>*</code>	<code>3 * 4 = 12</code>
Division	<code>/</code>	<code>5 / 2 = 2.5</code>
Integer Quotient	<code>//</code>	<code>5 // 2 = 2</code>
Integer Remainder	<code>%</code>	<code>5 % 2 = 1</code>

Arithmetic Expressions

We can chain these operators together to create whatever **expressions** we want! They can be as complex or as simple as you would imagine.

These **arithmetic expressions** follow the standard order of operations.

For example:

$$\begin{aligned}5 + 5 * 2 &= 15 \\ (5 + 5) * 2 &= 20\end{aligned}$$

Variables

Expressions allow us to perform calculations but we usually want to store/save the results of those calculations.

Additionally, we may want to use placeholders in our expressions to represent where outside data might make the results of our expressions *vary*.

Variables allow us to define containers that store values within them.

We can define a variable by specifying it's name to be followed by an `=` and an expression.

```
temperature = 74 , name = "Chris"
```

```
answer = 5 + 3 / 2
```

All variables need their own unique name in Python and names are case-sensitive.

Variable Naming Rules

All Python variable names must follow these rules:

- All variable names must begin with a letter or underscore
- Variable names can only contain letters, numbers, and underscores

Let's look at some examples:

- `hello` , `_hello` , `hello_world` 
- `hello world`  Variable names cannot contain spaces
- `Student1` , `student_1` , `first_student` 
- `1_student`  Variable names cannot start with a number
- `student#1`  Variable names cannot contain #'s

The need to output

Having to "think like a computer" gets a little annoying...

For example, suppose we wanted the computer to calculate something. To you and me, it is part of the assumed context that - if I ask you to calculate something for me - I also want you to tell me the answer.

However, computers don't understand any type of context.

When we want the computer to do something **AND** tell us the result, we need to explicitly tell them that!

```
1 5 + 5 # Produces no output?
```



No output received

The `print` Statement

A very common way we visualize the results of a program is using `print()` statements.

The `print()` statement allows us to tell the computer to output a line of text to the terminal.

Let's see how adding a `print()` statement makes a difference:

```
1 print(5 + 5) # Tells the computer to "Calculate 5 + 5 and then print the result"
```



```
10
```

Printing Expressions

A `print()` statement takes in zero or more expressions separated by a comma in its `()` and displays them as text in the terminal. Remember that an **expression** is anything that evaluates to a **value**.

Try changing the statements below to see how it changes the results:

```
1 print(5 + 5)
2 print("Hello world!")
3
4 answer = 7 - 2
5 print("The answer is", answer)
6
7 print("It is currently", 64, "degrees outside!")
```

```
10
Hello world!
The answer is 5
It is currently 64 degrees outside!
```